



“All your ROMs are Belong to us!”

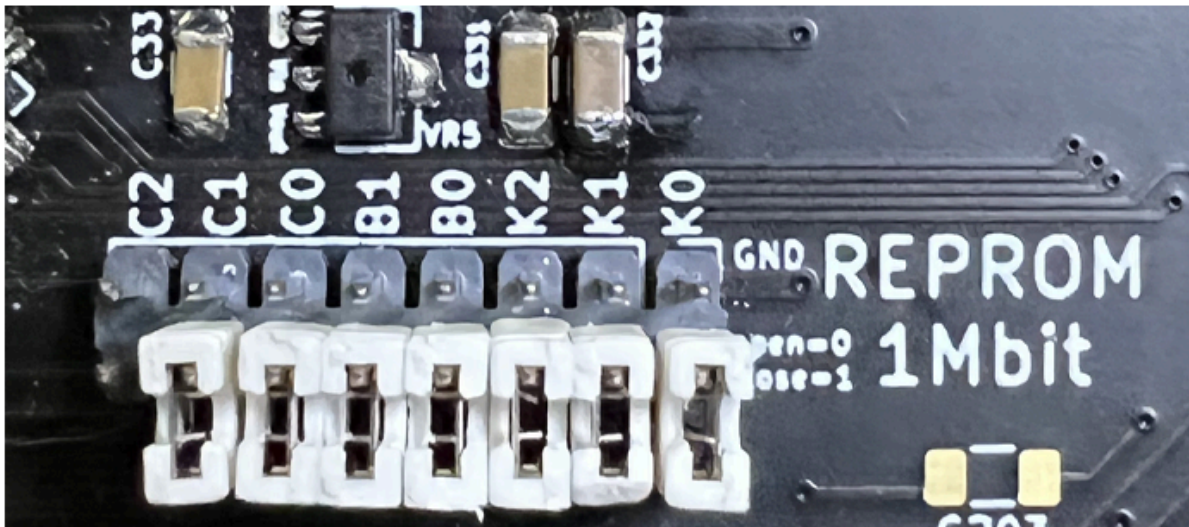
EVO takes a unique approach towards allowing you to manage all of your system ROMS images and Character sets, by placing them into single-unified EEPROM. A variety of chip models in a variety of sizes are supported, from the W27C010 @ 128KB, up to the 27C801 @ 1MB. Your choice of chip depends on the number of ROM images you'd like to have onboard and available for handy access.

Inspiration for the switchable multi-ROM approach that EVO is using came from the [RePROM64](#), which was originally designed by Henning Liebenau, however the implementation has been *heavily* modified for EVO, in that it's fully integrated into the [QAPLA \(EVO's Custom Developed PLA\)](#).

The extra Kernal slots allow you to bring along your own custom ROMS (e.g., [JiffyDOS](#), [JaffyDOS](#), [SpeedDOS](#), etc.), so you can experiment with blazing-fast load times, predefined Function Keys and other extended features. You can also change your character sets to suit your mood or to match your system-specific build theme.

EVO has other tricks up its sleeve too. You can actually store your favorite **Cartridge image files** on the same EEPROM if you like. This gives you on-the-fly access to system diagnostics, utilities and even games, *instantly*, by simply moving a couple jumper caps*.

Accessing different ROM images using jumpers



Enabling access to Integrated Cartridge ROMs

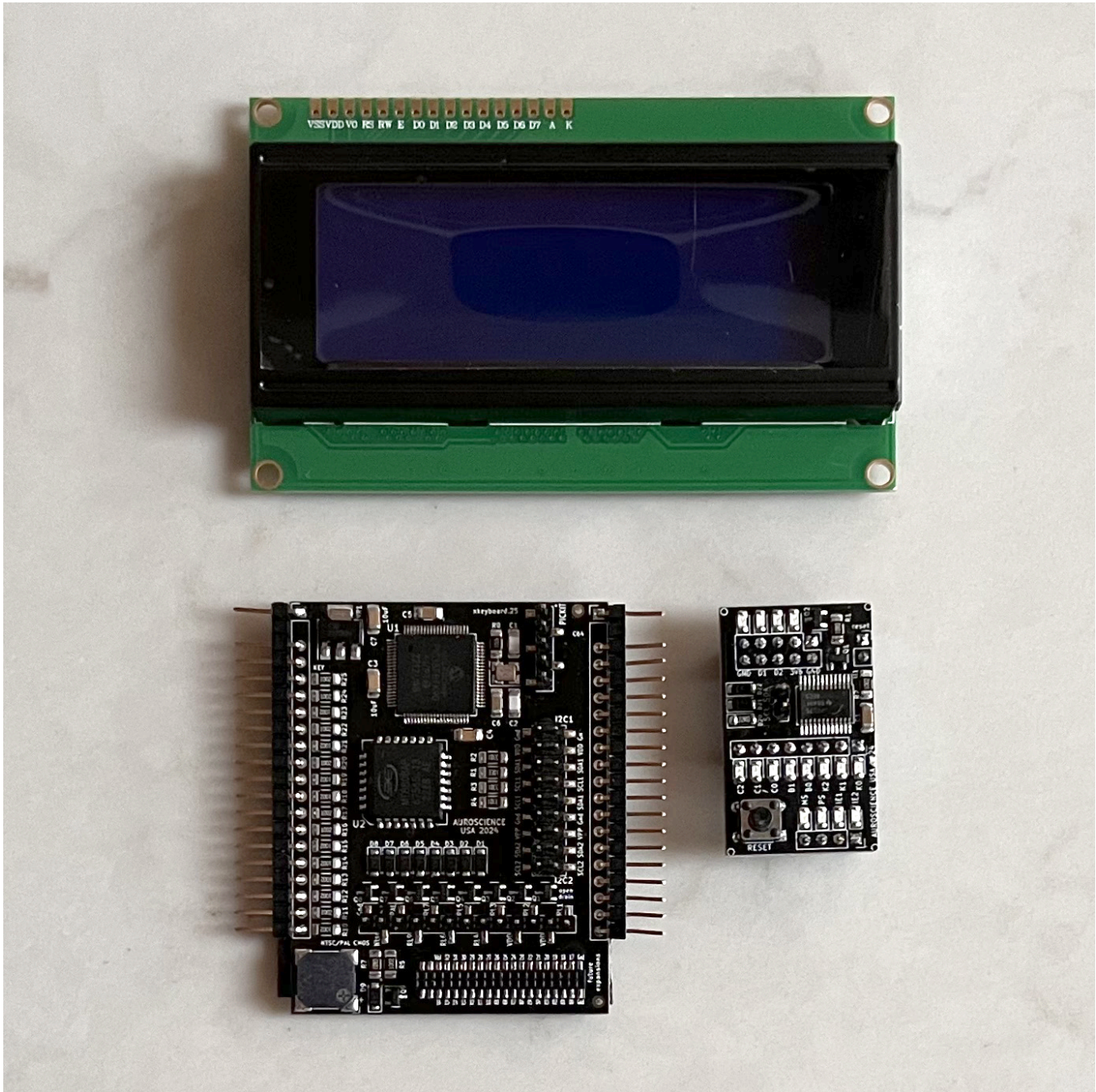


Enable Cartridge ROMs



Enable ROM Images

***IMPORTANT NOTE:** The system must be powered off before you attempt to move any of the jumper caps with your hand. However, you can completely automate the process of changing EVO configuration settings with the [EVO's Multi-Switch Module \(MSM\)](#)



Accessing your ROM images & Cart Files

The table below provides an outline of the different EEPROM types that are supported by EVO, along with the amount of ROM images and cartridge files that each can hold, as well as the memory addresses that each ROM type should occupy and the jumper settings that are required to access them.

Please note that the smallest EEPROM the EVO requires is the W27C010, which supports 128KB of ROM images. This provides access to 8x Kernal ROMs, 4x versions of Basic and 8x Character sets, but it will not have room left for any Cartridge files.

If you decide to GO BIG with the 27C801, with support for up to 1MB, you'll have enough room for all the aforementioned ROM images, as well as 64x cartridge files, which can either be 8KB or 16KB each.

Use the link below to access and make a copy of the memory map sheet, so you can plan out your own EEPROM image layout:

[EVO64 EPROM Memory Map](#)

Supported EEPROM(S)		ROM Type	ROM Count	ROM Images	ROM Size (KB)	Cumulative Size (KB)	EPROM Mem Addr.	JTAG CRT EN	Re-PROM Jumpers								
									C2	C1	C0	B1	B0	K2	K1	K0	
W27C010	W27C020	AM29F040B / W27C040	27C801	Kernel	1	Standard (901227)	8	8	\$00000 - \$01FFF	00	-	-	-	-	0	0	0
					2	JiffyDOS 6.01	8	16	\$02000 - \$03FFF	00	-	-	-	-	0	0	1
					3	JiffyDOS Modified	8	24	\$04000 - \$05FFF	00	-	-	-	-	0	1	0
					4	OpenROM Kernal	8	32	\$06000 - \$07FFF	00	-	-	-	-	0	1	1
				5	Standard (901227)	8	40	\$08000 - \$09FFF	00	-	-	-	-	1	0	0	
				6	JiffyDOS 6.01	8	48	\$0A000 - \$0BFFF	00	-	-	-	-	1	0	1	
				7	JiffyDOS Modified	8	56	\$0C000 - \$0DFFF	00	-	-	-	-	1	1	0	
				8	OpenROM Kernal	8	64	\$0E000 - \$0FFFF	00	-	-	-	-	1	1	1	
Basic	1	Standard (901226)	8	72	\$10000 - \$11FFF	00	-	-	-	0	0	-	-	-			
	2	Std Multiply Fix	8	80	\$12000 - \$13FFF	00	-	-	-	0	1	-	-	-			
	3	OpenROM Basic	8	88	\$14000 - \$15FFF	00	-	-	-	1	0	-	-	-			
	4	Standard (901226)	8	96	\$16000 - \$17FFF	00	-	-	-	1	1	-	-	-			
Charset	1	Standard (901225)	4	100	\$18000 - \$18FFF	00	0	0	0	-	-	-	-	-			
	2	C16/Plus4 Chargen	4	104	\$19000 - \$19FFF	00	0	0	1	-	-	-	-	-			
	3	PXL font 88665b	4	108	\$1A000 - \$1AFFF	00	0	1	0	-	-	-	-	-			
	4	MSX Chargen	4	112	\$1B000 - \$1BFFF	00	0	1	1	-	-	-	-	-			
	5	Atari 800 Chargen	4	116	\$1C000 - \$1CFFF	00	1	0	0	-	-	-	-	-			
	6	Amiga 1200	4	120	\$1D000 - \$1DFFF	00	1	0	1	-	-	-	-	-			
	7	OpenROM Chargen	4	124	\$1E000 - \$1EFFF	00	1	1	0	-	-	-	-	-			
	8	Standard (901225)	4	128	\$1F000 - \$1FFFF	00	1	1	1	-	-	-	-	-			

Organizing your own EEPROMs

If you don't mind rolling up your DIY sleeves, you can program your own EEPROMs for EVO, by using a TL866II or similar programmer. These are readily available on Amazon, Bay, AliEx and elsewhere, and they're *generally* fairly inexpensive (all depends on current supply & demand).

The software interfaces for some programmers out there can be a little clumsy, but like everything else, they get much easier to use once you've felt your way through programming a couple ROMs.

As a frame of reference, you can use the steps on the following pages to complete the programming of your EEPROM(s), using a typical TL866II programmer (or clone).

Programming with the TL866II / T48

Step 1: Download and install the latest version of the programming software from the XGecu website:
<http://xgecu.com/EN/Download.html>

The site may initially load in Chinese, but don't worry, a recent browser version *should* be able to perform a live translation. There should also be an option in the upper right hand corner of the loaded page to switch the language to English.

The programmer software is generally only available for Windows systems, although we've also had success using it from a Windows Virtual Machine running on our Mac systems, so this is an option as well.

Technical Support:
532007590@QQ.com

Chinese | English
   

XGecu® 鑫工电子

[Home](#) | [XGecu Programmer](#) | **Download** | [Forum](#) | [Contact Us](#)



XGecu T48 Universal programmer
Support: 32086+ (31/10/2022) continue
SIZE: 10cm x 6.5cm x 2.8cm
Super performance/Super speed/ Super small portable/ Cool appearance/ Low power consumption / Super stability


www.HKLRF.com



Download
Documentation . Software and Data Sheet

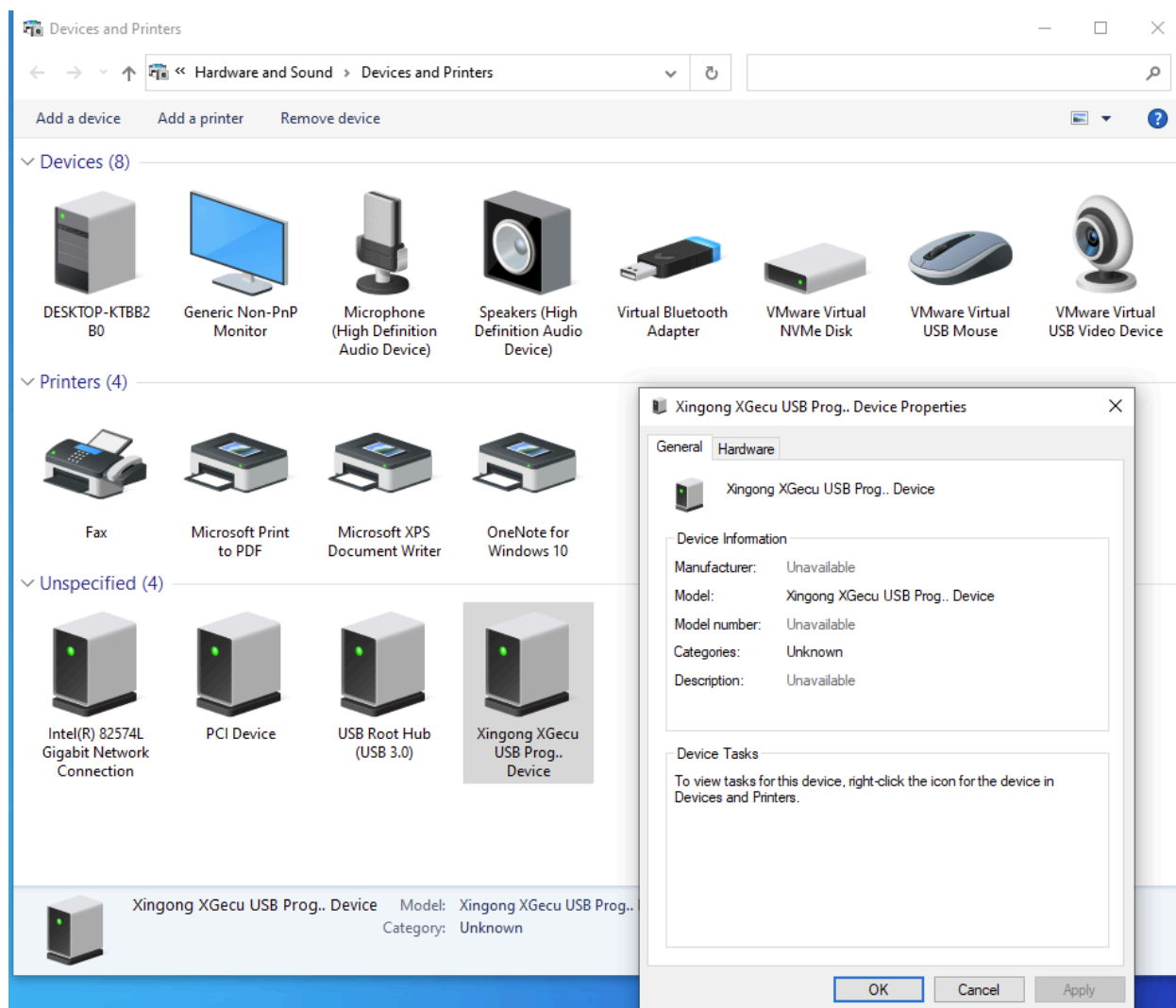
About XGecu T48/T56/TL866II programmer

Content	Overseas	Local
T48/T56/TL866II Plus Programmer Application Software (Chinese/English)		
T56 Plus IC Supported List		
T48 Plus IC Supported List		
TL866II Plus IC Supported List		
T48/T56/TL866II Plus Programmer Universal Manual (English)		
T48 PLCC44 Adapter DIY(for C51 / 27Cxxx /PIC)		
T48 DIP42(PLCC44) Adapter DIY for 27C(V)800/160/322 MX27C8100/1610		

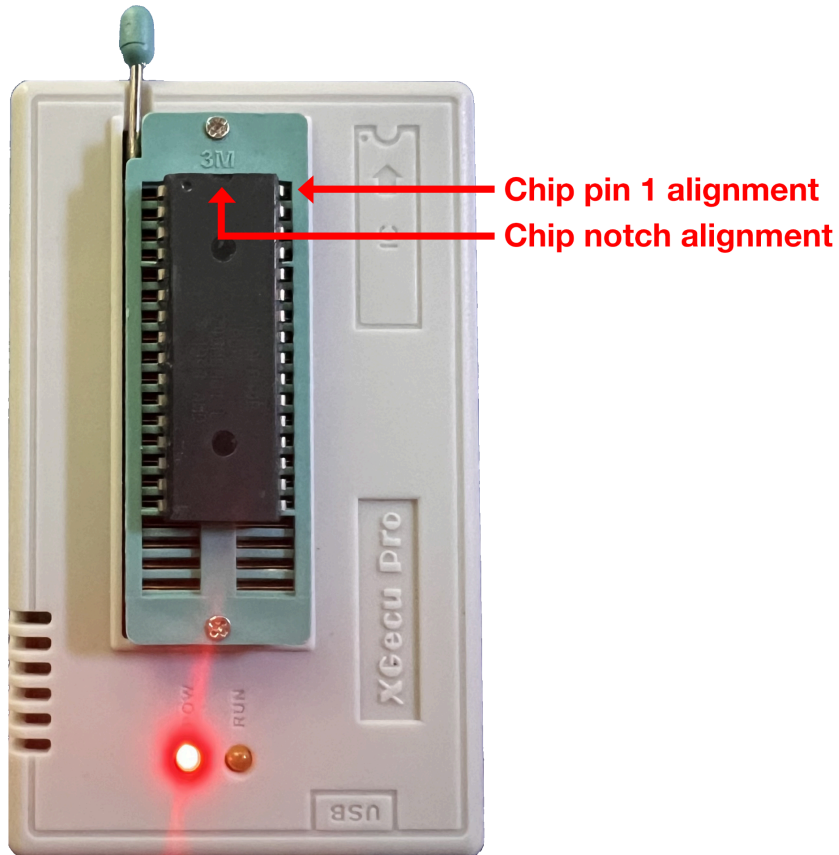
About TL866 programmer

Content	Download	Note
TL866 Programmer Application Software (Chinese/English V6.85)		
TL866 IC Supported List		
TL866 Programmer Universal Manual (English) by yanyan.816@163.com		sunwenjun

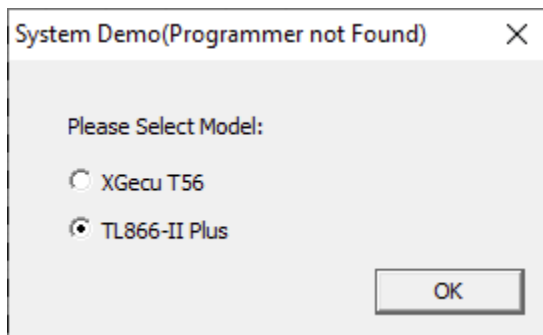
Step 2: Connect your programmer to the USB port of the desktop system that you'll be running doing the programming from. The system should auto-detect it as a USB device.



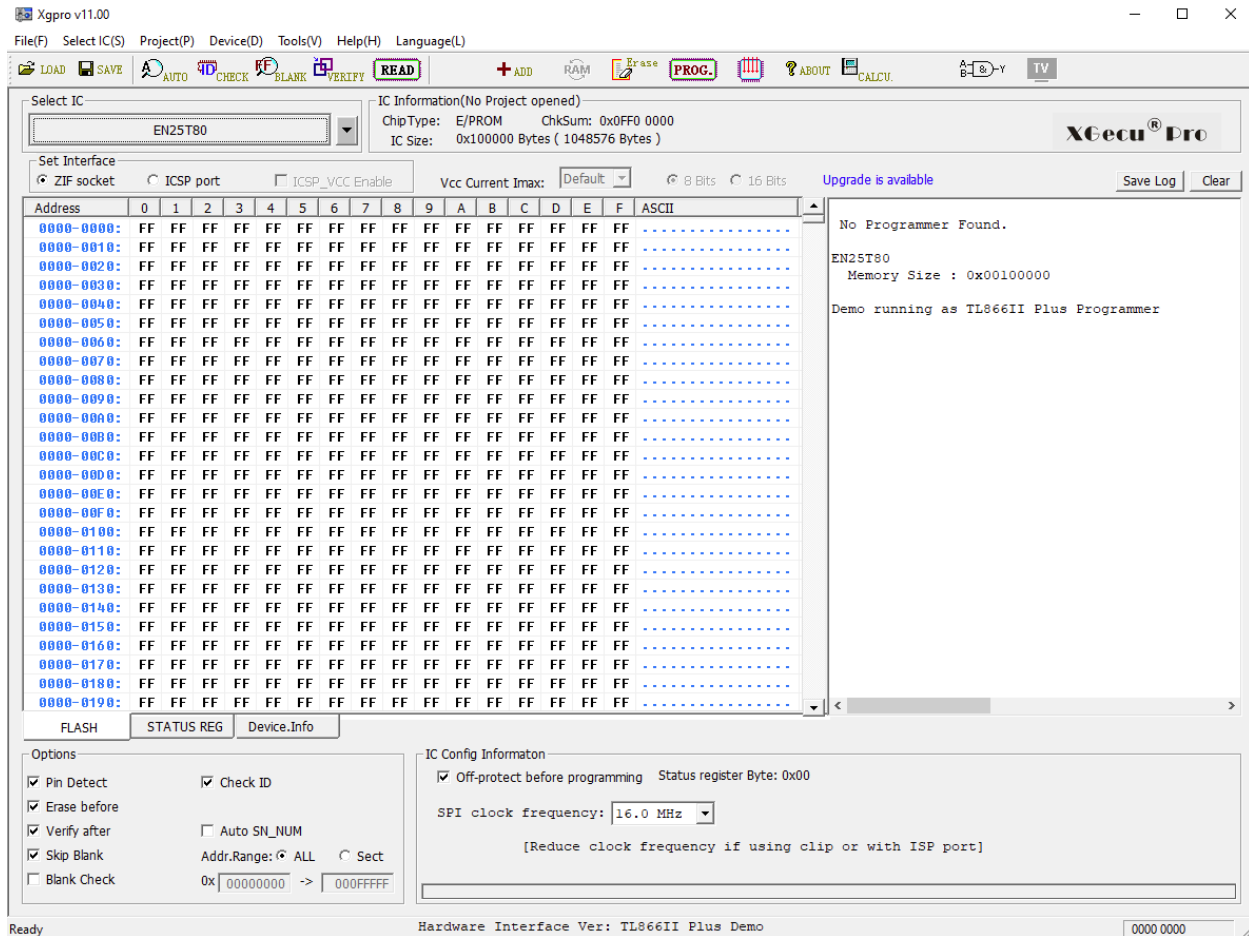
Step 3: Place your programmable ROM into the Programmer, using the correct chip orientation



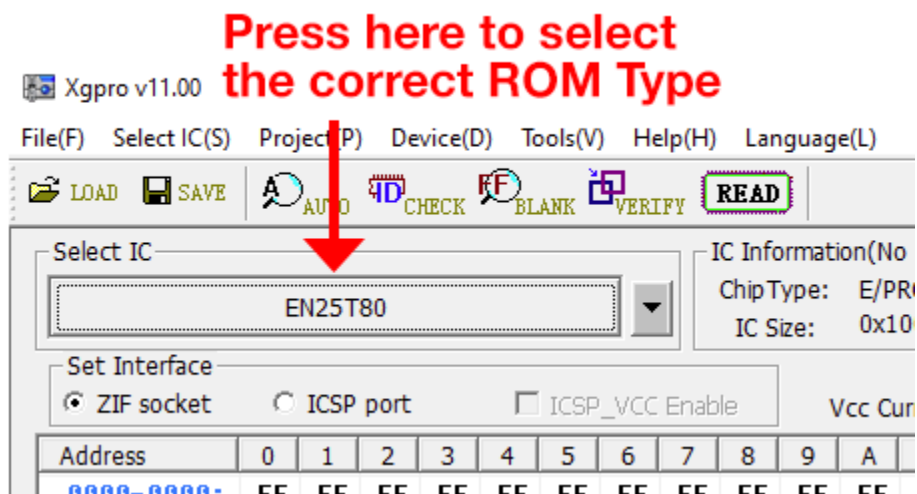
Step 4: Launch the XGpro application and select the correct Programmer type



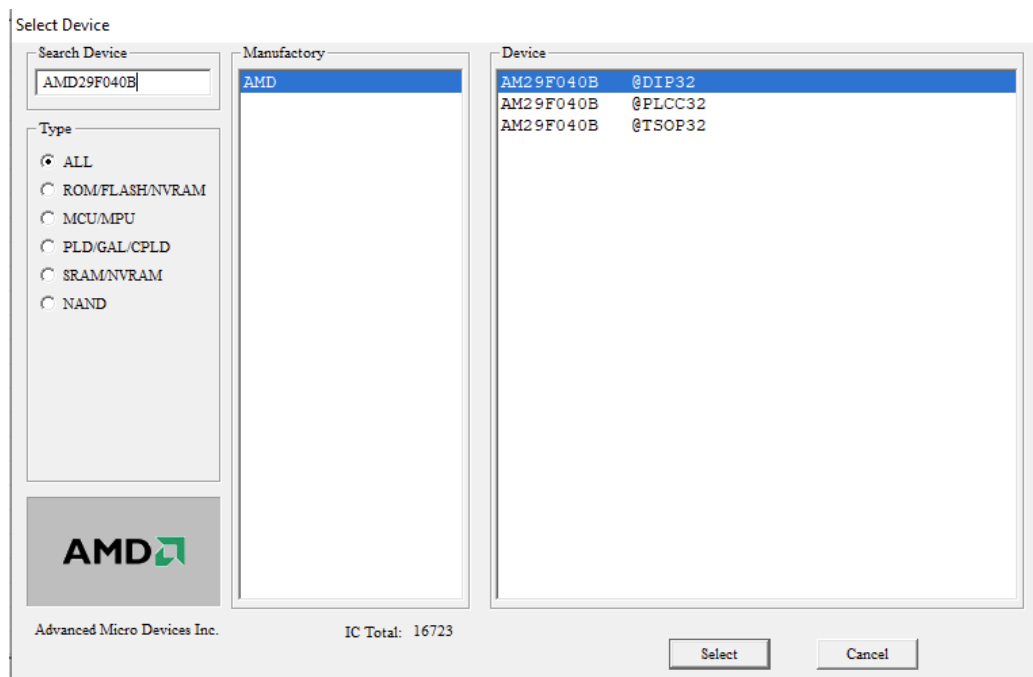
Step 5: Confirm the Initial Application Ready State



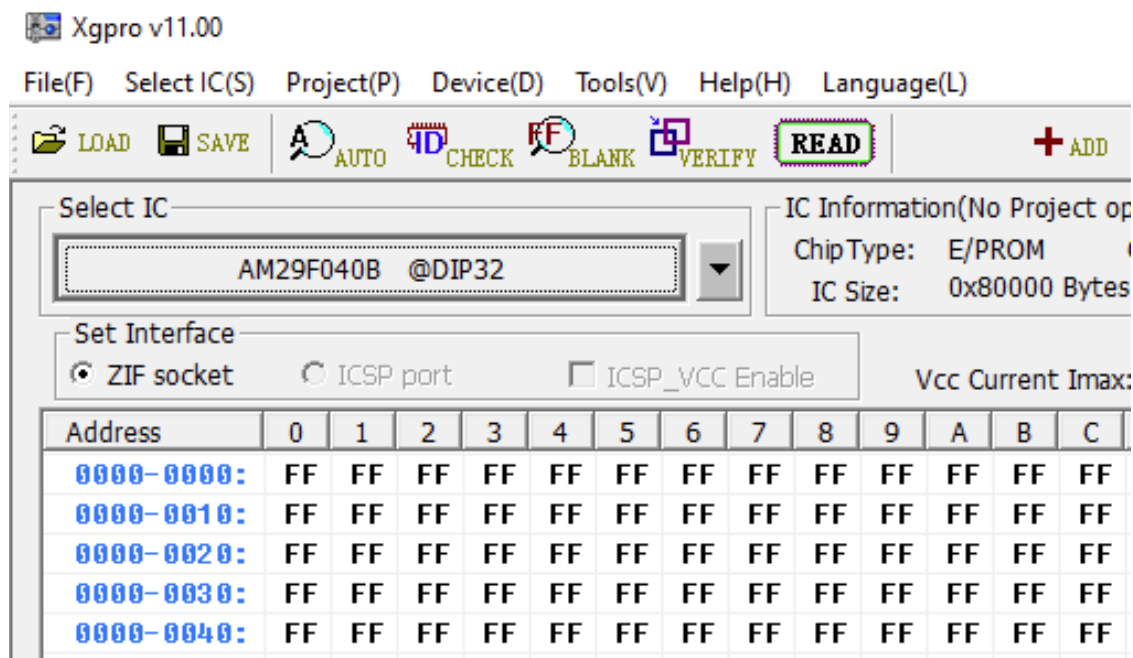
Step 6: Press the “Select IC” button to Change the EPROM Type



Step 7: Choose the correct programmable ROM chip type, using the “Search Device” field in the top right of the “Select Device” pop-up dialog box. If you bought the optional EVO Through-Hole parts BOM, it will have most likely come with an **AMD29F040B**



Step 8: Ensure that the correct ROM type now appears on the “Select IC” Button



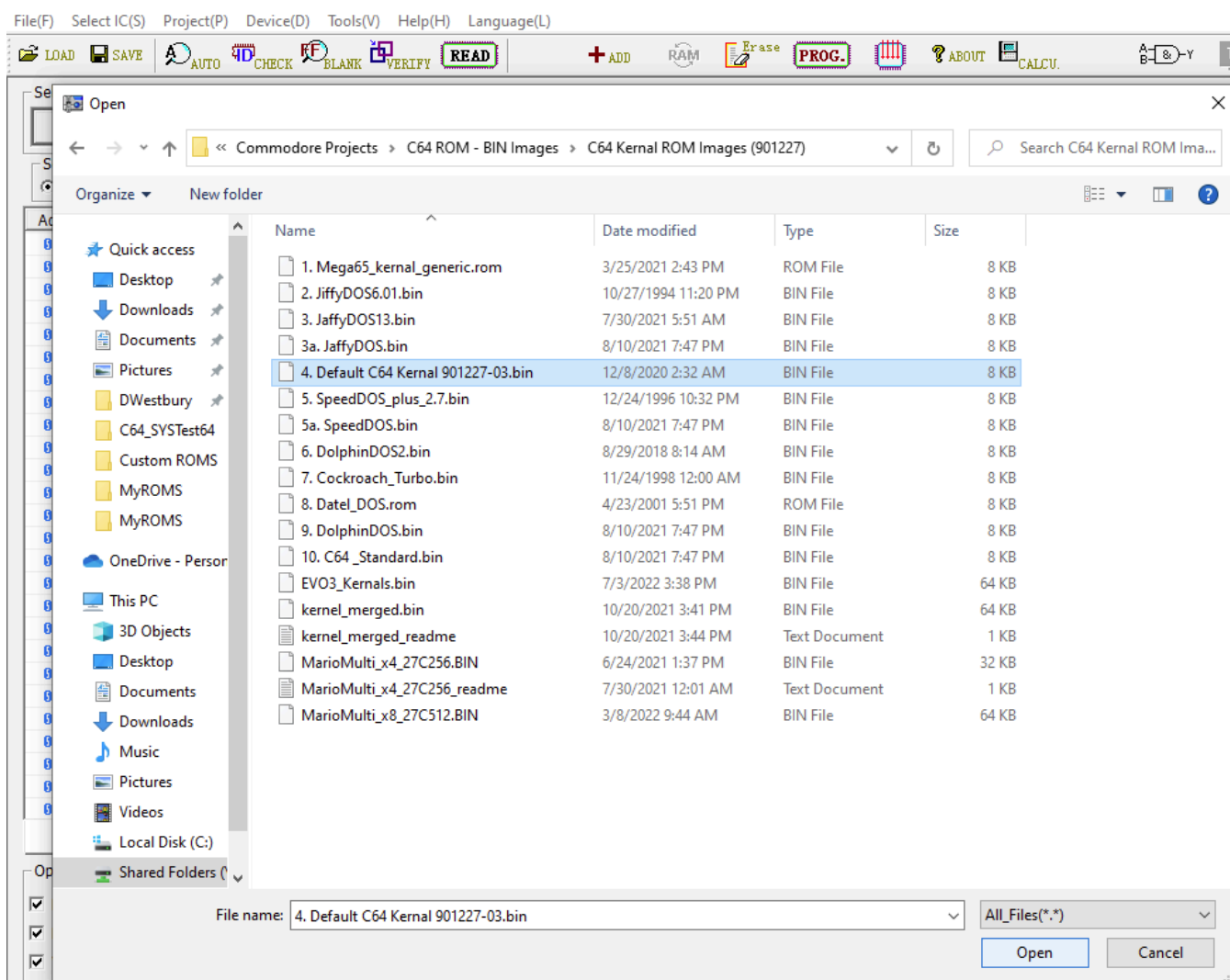
Step 9: Refer to the EVO EEPROM memory map chart on the last page of this document to familiarize yourself with the memory locations for each of the system ROM types

Examples:

ROM Type	ROM Count	ROM Images	ROM Size (KB)	Cumulative Size (KB)	EPROM Mem Addr.
Kernel	1	Standard (901227)	8	8	\$00000 - \$01FFF
	2	JiffyDOS 6.01	8	16	\$02000 - \$03FFF
	3	JiffyDOS Modified	8	24	\$04000 - \$05FFF
	4	OpenROM Kernel	8	32	\$06000 - \$07FFF

Step 10: Prepare to load the standard C64 Kernel ROM (901227) into the first available memory slot (\$00000 - \$01FFF), then press the “Load” button in the upper left corner of the main application to find the required ROM file.

NOTE: Commodore ROMs can typically be found on the internet and/or from the files that come with a copy of the VICE C64 emulator



Step 11: Take note of the default settings in the “File load Options” pop-up window.

File Format: **Binary**

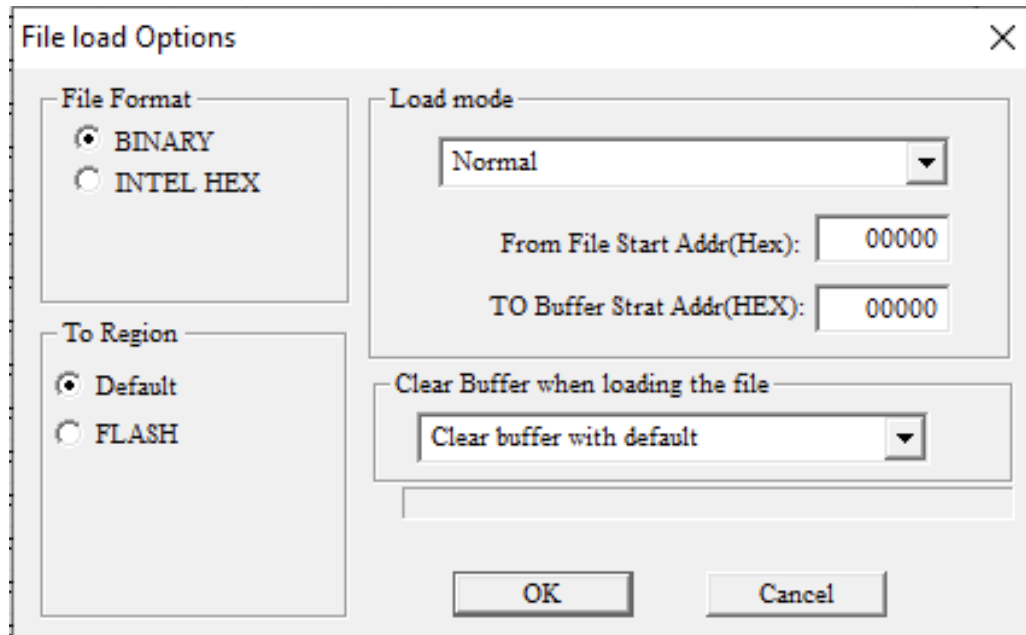
To Region: **Default**

Load Mode: **Normal**

From File Start Addr(Hex): **00000**

TO Buffer Start Addr(HEX): **00000**

Clear Buffer when Loading the file: **Clear buffer with default**



The image shows a Windows-style dialog box titled "File load Options" with a close button (X) in the top right corner. The dialog is divided into several sections:

- File Format:** Contains two radio buttons. "BINARY" is selected (indicated by a filled circle), and "INTEL HEX" is unselected (indicated by an empty circle).
- To Region:** Contains two radio buttons. "Default" is selected (indicated by a filled circle), and "FLASH" is unselected (indicated by an empty circle).
- Load mode:** A dropdown menu showing "Normal".
- From File Start Addr(Hex):** A text input field containing "00000".
- TO Buffer Start Addr(HEX):** A text input field containing "00000".
- Clear Buffer when loading the file:** A dropdown menu showing "Clear buffer with default".

At the bottom of the dialog, there are two buttons: "OK" and "Cancel".

Step 12: Look at the ASCII section of the main application window. You should now see evidence that the ROM file has been “pre-staged” to be loaded into the correct ROM file location on the ROM chip

Select IC

AM29F040B @DIP32

IC Information(No Project opened)
Chip Type: E/PROM ChkSum: 0x07E7 E70A
IC Size: 0x80000 Bytes (524288 Bytes)

Set Interface
☒ ZIF socket ☐ ICSP port ☐ ICSP_VCC Enable

Vcc Current Imax: Default ☒ 8 Bits ☐ 16 Bits

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	ASCII
0000-0000:	85	56	20	0F	BC	A5	61	C9	88	90	03	20	D4	BA	20	CC	.U ...a....
0000-0010:	BC	A5	07	18	69	81	F0	F3	38	E9	01	48	A2	05	B5	69	i...8..H...i
0000-0020:	B4	61	95	61	94	69	CA	10	F5	A5	56	85	70	20	53	B8	.a.a.i....U.p S.
0000-0030:	20	B4	BF	A9	C4	A0	BF	20	59	E0	A9	00	85	6F	68	20	Y....oh
0000-0040:	B9	BA	60	85	71	84	72	20	CA	BB	A9	57	20	28	BA	20	..`.q.r ...W (.
0000-0050:	5D	E0	A9	57	A0	00	4C	28	BA	85	71	84	72	20	C7	BB	]..W..L(..q.r ..
0000-0060:	B1	71	85	67	A4	71	C8	98	D0	02	E6	72	85	71	A4	72	.q.g.q.....r.q.r
0000-0070:	20	28	BA	A5	71	A4	72	18	69	05	90	01	C8	85	71	84	(..q.r.i.....q.
0000-0080:	72	20	67	B8	A9	5C	A0	00	C6	67	D0	E4	60	98	35	44	r g..\...g..\`5D
0000-0090:	7A	00	68	28	B1	46	00	20	2B	BC	30	37	D0	20	20	F3	z.h(.F. +.07. .
0000-00A0:	FF	86	22	84	23	A0	04	B1	22	85	62	C8	B1	22	85	64	..".#..."b..."d
0000-00B0:	A0	08	B1	22	85	63	C8	B1	22	85	65	4C	E3	E0	A9	8B	..."c..."eL....
0000-00C0:	A0	00	20	A2	BB	A9	8D	A0	E0	20	28	BA	A9	92	A0	E0	 (.
0000-00D0:	20	67	B8	A6	65	A5	62	85	65	86	62	A6	63	A5	64	85	g...e.b.e.b.c.d.
0000-00E0:	63	86	64	A9	00	85	66	A5	61	85	70	A9	80	85	61	20	c.d...f.a.p...a
0000-00F0:	D7	B8	A2	8B	A0	00	4C	D4	BB	C9	F0	D0	07	84	38	86	L.....8.
0000-0100:	37	4C	63	A6	AA	D0	02	A2	1E	4C	37	A4	20	D2	FF	B0	7Lc.....L7. ...
0000-0110:	E8	60	20	CF	FF	B0	E2	60	20	AD	E4	B0	DC	60	20	C6	`.....`.....`.
0000-0120:	FF	B0	D6	60	20	E4	FF	B0	D0	60	20	8A	AD	20	F7	B7	...`.....`.....`.
0000-0130:	A9	E1	48	A9	46	48	AD	0F	03	48	AD	0C	03	AE	0D	03	..H.FH...H.....
0000-0140:	AC	0E	03	28	6C	14	00	08	8D	0C	03	8E	0D	03	8C	0E	...(1.....
0000-0150:	03	68	8D	0F	03	60	20	D4	E1	A6	2D	A4	2E	A9	2B	20	.h...`.....-...+
0000-0160:	D8	FF	B0	95	60	A9	01	2C	A9	00	85	0A	20	D4	E1	A5	`.....
0000-0170:	0A	A6	2B	A4	2C	20	D5	FF	B0	57	A5	0A	F0	17	A2	1C	..+., ...W.....
0000-0180:	20	B7	FF	29	10	D0	17	A5	7A	C9	02	F0	07	A9	64	A0	..).....z.....d.
0000-0190:	A3	4C	1E	AB	60	20	B7	FF	29	BF	F0	05	A2	1D	4C	37	.L...`.....).....L7

FLASH Device.Info

Step 13: Using the scrollbar on the right-hand side of the ASCII section of the main application window, scroll down to memory location \$01fff to confirm that the pre-staged ROM file image has been loaded completely.

Beginning at memory location \$02000 you should still see empty memory space, which is indicated by the “dots”

Select IC

AM29F040B @DIP32

IC Information(No Project opened)
Chip Type: E/PROM ChkSum: 0x07E7 E70A
IC Size: 0x80000 Bytes (524288 Bytes)

Set Interface
☒ ZIF socket ☐ ICSP port ☐ ICSP_VCC Enable

Vcc Current Imax:

Default

☒ 8 Bits ☐ 16 Bits

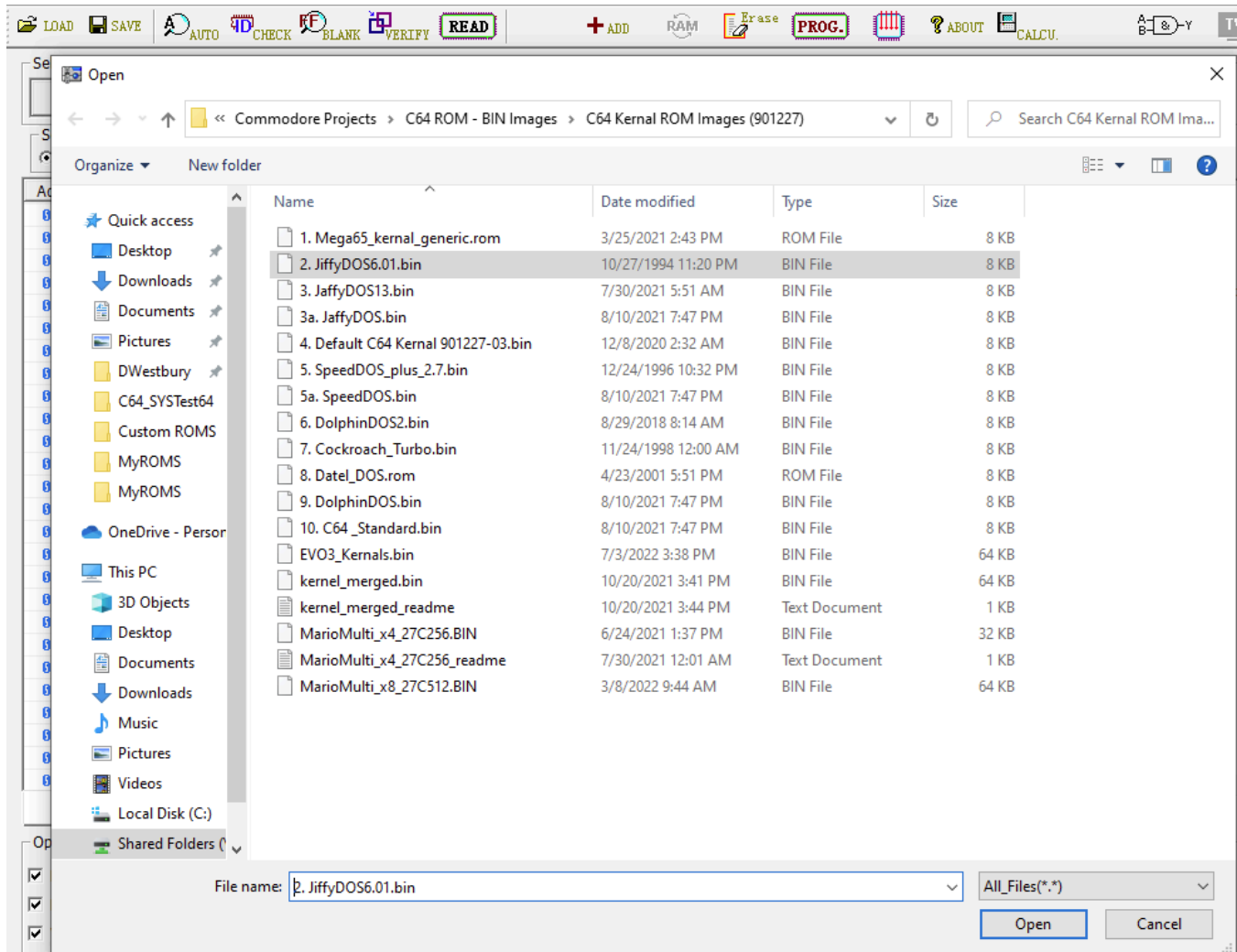
Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	ASCII
0000-1FC0:	6C	1A	03	6C	1C	03	6C	1E	03	6C	20	03	6C	22	03	6C	1..1..1..1 .1".1
0000-1FD0:	24	03	6C	26	03	4C	9E	F4	4C	DD	F5	4C	E4	F6	4C	DD	\$.1&.1..1..1..1..1
0000-1FE0:	F6	6C	28	03	6C	2A	03	6C	2C	03	4C	9B	F6	4C	05	E5	.1(.1*.1,.1..1..1..1
0000-1FF0:	4C	0A	E5	4C	00	E5	52	52	42	59	43	FE	E2	FC	48	FF	L..L..RRBYC...H.
0000-2000:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2010:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2020:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2030:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2040:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2050:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2060:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2070:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2080:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2090:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-20A0:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-20B0:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-20C0:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-20D0:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-20E0:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-20F0:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2100:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2110:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2120:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2130:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2140:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0000-2150:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	

FLASH Device.Info

Step 14: Reference the EVO EEPROM memory map again to take note of the memory location for the *next* ROM file image to be prestaged. In this case Kernal ROM #2 will be loaded into memory location \$02000

ROM Type	ROM Count	ROM Images	ROM Size (KB)	Cumulative Size (KB)	EPROM Mem Addr.
mal	1	Standard (901227)	8	8	\$00000 - \$01FFF
	2	JiffyDOS 6.01	8	16	\$02000 - \$03FFF
	3	JiffyDOS Modified	8	24	\$04000 - \$05FFF
	4	OpenROM Kernal	8	32	\$06000 - \$07FFF

Step 15: Now you will use the file load button in the main application window to add the next ROM file image. The process is a bit different from here, so please take note of the next details



Step 16: Configure the “File load Options” for the next ROM image, so it will be prestaged into into the next available memory location. Please note the important differences highlighted in the pop-up dialog box

File Format: **Binary**

To Region: **Default**

Load Mode: **Normal**

From File Start Addr(Hex): **00000**

TO Buffer Start Addr(HEX): **02000**

Clear Buffer when Loading the file: **Disable**

File load Options

File Format

- ☒ BINARY
- ☐ INTEL HEX

To Region

- ☒ Default
- ☐ FLASH

Load mode

Normal

From File Start Addr(Hex): 00000

TO Buffer Strat Addr(HEX): 02000

Clear Buffer when loading the file

Disable

OK Cancel

Step 17: Look at memory location \$02000 in the ASCII section of the main application window. You should see evidence that the second ROM image has been loaded (e.g., the dots have been replaced by characters)

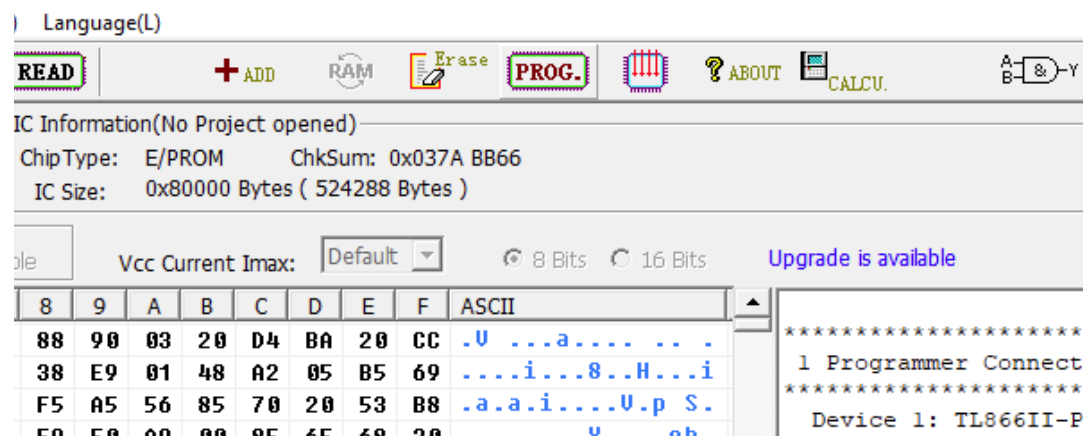
Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	ASCII
0000-1FC0:	6C	1A	03	6C	1C	03	6C	1E	03	6C	20	03	6C	22	03	6C	1..1..1..1 .1".1
0000-1FD0:	24	03	6C	26	03	4C	9E	F4	4C	DD	F5	4C	E4	F6	4C	DD	\$.1&.L..L..L..L.
0000-1FE0:	F6	6C	28	03	6C	2A	03	6C	2C	03	4C	9B	F6	4C	05	E5	.1(.1*.1,.L..L..
0000-1FF0:	4C	0A	E5	4C	00	E5	52	52	42	59	43	FE	E2	FC	48	FF	L..L..RRBYC...H.
0000-2000:	85	56	20	0F	BC	A5	61	C9	88	90	03	20	D4	BA	20	CC	.U ...a.... . .
0000-2010:	BC	A5	07	18	69	81	F0	F3	38	E9	01	48	A2	05	B5	69	i...8..H...i
0000-2020:	B4	61	95	61	94	69	CA	10	F5	A5	56	85	70	20	53	B8	.a.a.i....U.p S.
0000-2030:	20	B4	BF	A9	C4	A0	BF	20	59	E0	A9	00	85	6F	68	20	Y....oh
0000-2040:	B9	BA	60	85	71	84	72	20	CA	BB	A9	57	20	28	BA	20	..`.q.r ...W (.
0000-2050:	5D	E0	A9	57	A0	00	4C	28	BA	85	71	84	72	20	C7	BB	]..W..L(..q.r ..
0000-2060:	B1	71	85	67	A4	71	C8	98	D0	02	E6	72	85	71	A4	72	.q.g.q.....r.q.r
0000-2070:	20	28	BA	A5	71	A4	72	18	69	05	90	01	C8	85	71	84	(..q.r.i.....q.
0000-2080:	72	20	67	B8	A9	5C	A0	00	C6	67	D0	E4	60	98	35	44	r g..\...g...`5D
0000-2090:	7A	00	68	28	B1	46	00	20	2B	BC	30	37	D0	20	20	F3	z.h(.F. +.07. .
0000-20A0:	FF	86	22	84	23	A0	04	B1	22	85	62	C8	B1	22	85	64	..".#..."b..."d
0000-20B0:	A0	08	B1	22	85	63	C8	B1	22	85	65	4C	E3	E0	A9	8B	..."c..."eL....
0000-20C0:	A0	00	20	A2	BB	A9	8D	A0	E0	20	28	BA	A9	92	A0	E0	 (.....
0000-20D0:	20	67	B8	A6	65	A5	62	85	65	86	62	A6	63	A5	64	85	g..e.b.e.b.c.d.
0000-20E0:	63	86	64	A9	00	85	66	A5	61	85	70	A9	80	85	61	20	c.d...f.a.p...a
0000-20F0:	D7	B8	A2	8B	A0	00	4C	D4	BB	C9	F0	D0	07	84	38	86	L.....8.
0000-2100:	37	4C	63	A6	A0	00	02	A2	1E	4C	37	A4	20	D2	FF	B0	7Lc.....L7. ...
0000-2110:	E8	60	20	CF	FF	B0	E2	60	20	AD	E4	B0	DC	60	20	C6	.`.....`.....`.
0000-2120:	FF	B0	D6	60	20	E4	FF	B0	D0	60	20	8A	AD	20	F7	B7	...`.....`.....`.
0000-2130:	A9	E1	48	A9	46	48	AD	0F	03	48	AD	0C	03	AE	0D	03	..H.FH...H.....
0000-2140:	AC	0E	03	28	6C	14	00	08	8D	0C	03	8E	0D	03	8C	0E	...(1.....
0000-2150:	03	68	8D	0F	03	60	20	D4	E1	A6	2D	A4	2E	A9	2B	20	.h...`.....-...+

Step 18: You will effectively repeat steps 14-17 over-and-over-again, until you've fully pre-staged all of the ROM images that you're interested in making available for your EVO system.

Please note that you do not need to have multiple Kernels, Basics and Character sets loaded, but **you will need at least ONE of each**, preferably loaded into the first memory slots for each ROM type

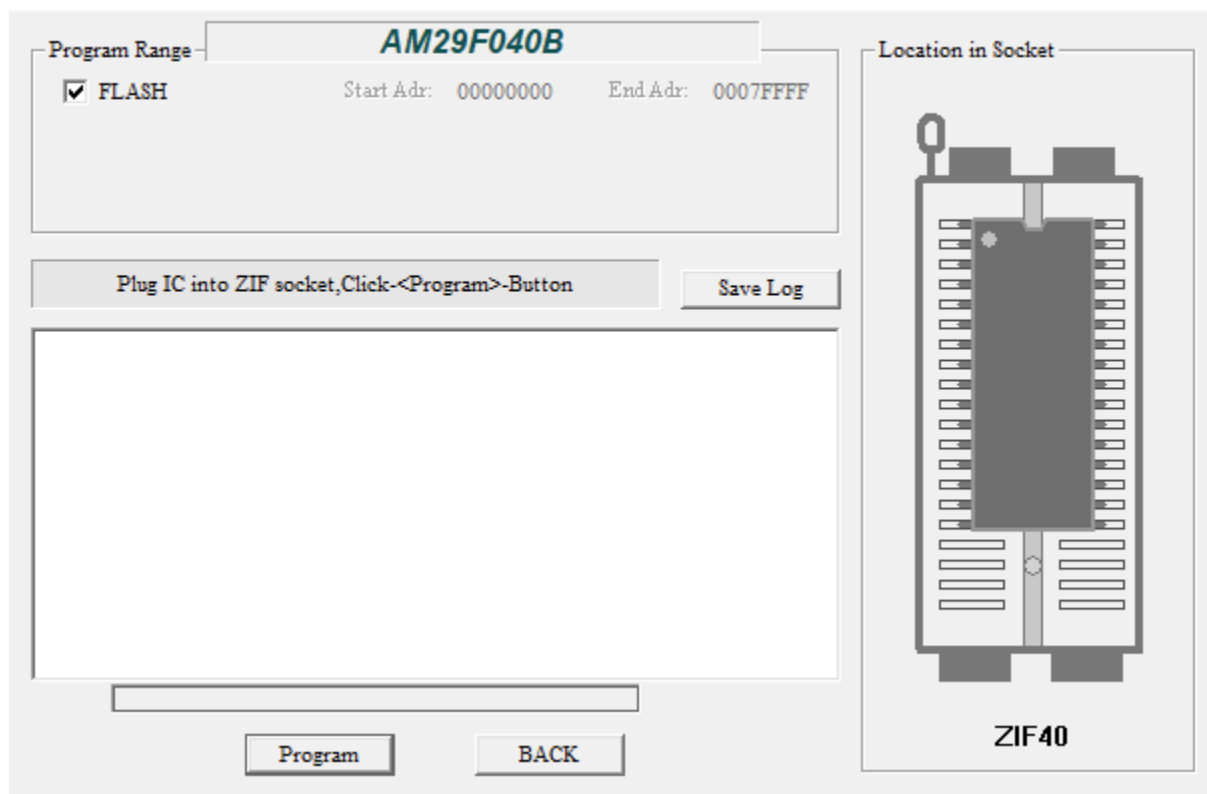
Kernal #1 - \$00000-\$01FFF
 Basic #1 - \$10000-\$11FFF
 Charset #1 - \$18000-\$18FFF

Step 19: Once all of your ROM images have been pre-staged, it's time to write (burn) them to your programmable ROM by pressing the “PROG” button in the middle of the top toolbar in the main application window.



Step 20: Press the “Program”button on the “Chip Program” pop-up window.

Chip Program APP Version: 11.00



Step 21: Monitor the programming progress

Chip Program APP Version: 11.00

Program Range

AM29F040B

☒ FLASH Start Adr: 00000000 End Adr: 0007FFFF

Programming FLASH ...

Save Log

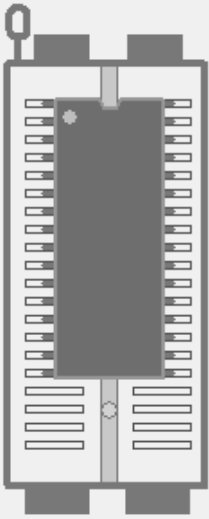
Pins Detected Passed!
ID: 0x 01 A4OK!
Erase ...Succeeded. Time : 5.672 S
Programming FLASH ...

48.43%

Program

BACK

Location in Socket



ZIF40

Step 22: Confirm that programming was successful

Chip Program APP Version: 11.00

Program Range

AM29F040B

☒ FLASH Start Adr: 00000000 End Adr: 0007FFFF

Programming ...Succeeded

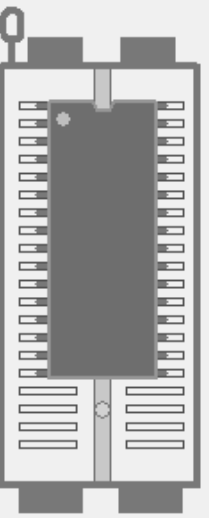
Save Log

Pins Detected Passed!
ID: 0x 01 A4OK!
Erase ...Succeeded. Time : 5.672 S
Programming FLASH ...Succeeded. Time : 20906ms
Verifying FLASH ...Succeeded. Time : 4016ms
Programming ...Succeeded

Program

BACK

Location in Socket



ZIF40

Step 23: Congratulations! Your ROM is now ready to be added into your EVO system!

Please note the orientation of the chip notch which is facing the rear of the EVO board (e.g., pointing towards the User, Dataset and cartridge ports, etc.)



Tools & Processes for ROM Management

SX64Man's Video on Multi-ROM Mgmt:

<https://www.youtube.com/watch?v=2JFnNYqYkGE>

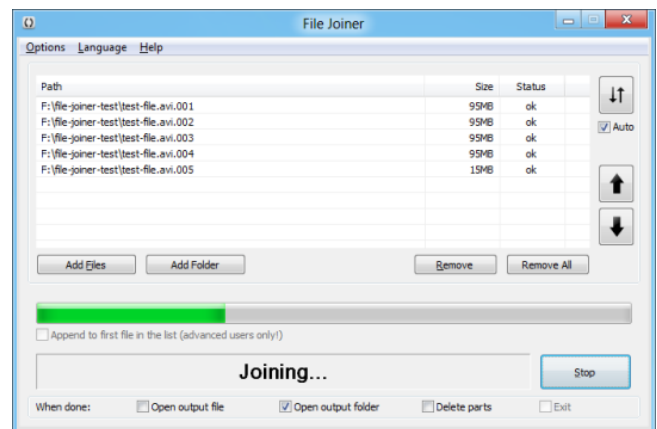
Our friend and fellow EVO64 beta tester, Mario the **SX64 man** has made an excellent video that steps through the process that he's using to combine multiple ROM & Cartridge images together, before writing them to the EEPROM.



The Igorware File Joiner Utility

<https://www.igorware.com/file-joiner/download>

In the **SX64Man** video above, there's references to this Windows utility, which enables you to easily join files together, before attempting to write them to your EPROM (watch the video for details)



C64 Kernal Disassembly Info:

<https://www.pagetable.com/c64ref/c64disasm/>

Info regarding ROM memory locations controlling boot-screen, side-borders and text colors, as well as the welcome message at the top of the screen

Ultimate Commodore 64 Reference 6502 KERNAL API ROM Disassembly Memory Map

C64 BASIC & KERNAL ROM Disassembly

by Michael Steil, github.com/mist64/c64ref. Revision 6ce14d7, 2022-01-23

This allows you to view different commentaries side-by-side. You can enable/disable commentaries.

☒	Microsoft/Commodore Source	The original M6502 BASIC source by M and the original C64 KERNAL source b
☒	Data Becker [German]	German-language comments from Das
☒	Lee Davison	Comments from The almost completely
☒	Bob Sander-Cederlof [BASIC only]	Comments adapted from S-C DocuMer
☒	Magnus Nyman [KERNAL only]	Comments from JIFFYDOS version 6.0 comments.
☒	Marko Mäkelä	Comments from the Commodore 64 B

Disassembly

..A000 94 E3

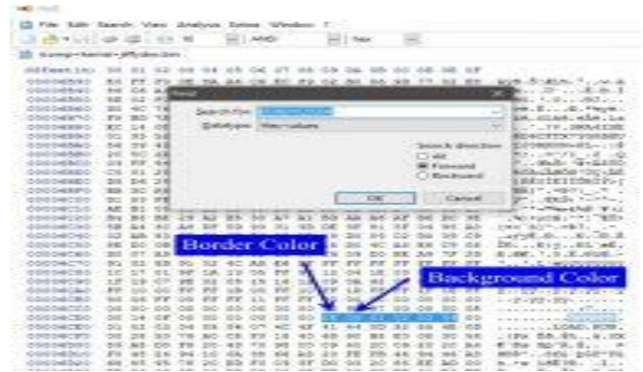
..A002 7B E3

Microsoft/Commodore Source

Breadbox64 C64 Kernel Modding:

<https://www.breadbox64.com/blog/c64-kernel-mods/>

Useful blog post covering the ins and outs of editing Kernel ROMs to modify colors and boot screen text



C64 Kernel Mod v1.2

https://tools.anemo.se/c64/kernal-mod/?fbclid=IwAR3XjF_6u6Ar2ihbr977_nBuHplfdqcANs8DxmjNGk5U-GLvEq0gQkK6-Ac

A useful online GUI tool that lets you load a Kernal image file, tweak the colors and boot text, then save the result to a file, which can be written to your EEPROM

C64 Kernel Mod v1.2

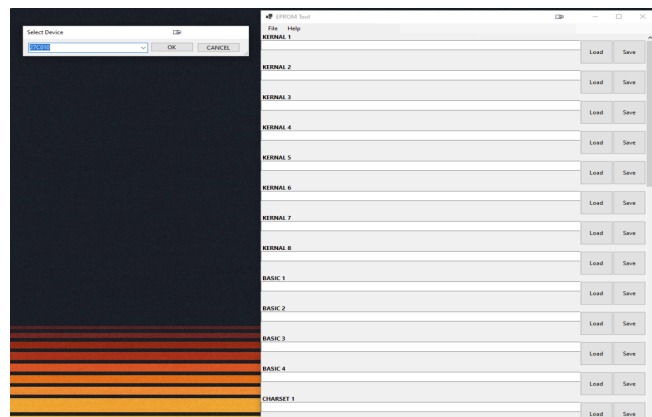


EPROM Tool v1.1

<https://drive.google.com/file/d/11YxtjMHBmiimlivL0H8MAqXKcpnHhDdX/view?usp=sharing>

This cool Windows-Only utility was developed by **Six of DLOC** and shared in the [EVO64 Discord](#), in the [#multi-rom-support channel](#) by **Dragos**

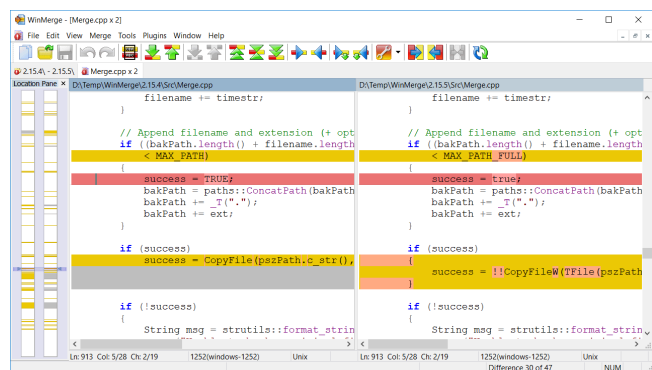
It's an EVO specific ROM / CRT file joiner that lets you choose your EPROM, then load your image files into a binary blob for easier writing.



WinMerge File Compare

<https://winmerge.org/?lang=en>

Want to see what's changed between two file versions? Then this handy Windows utility is your quick and handy go-to resource:



EVO ROM Image Mapping Reference

Here's [Direct link to a Google Sheet](#) with a detailed ROM Image map. You can download your own copy of this file to keep track of the ROM images that you're using on your system.

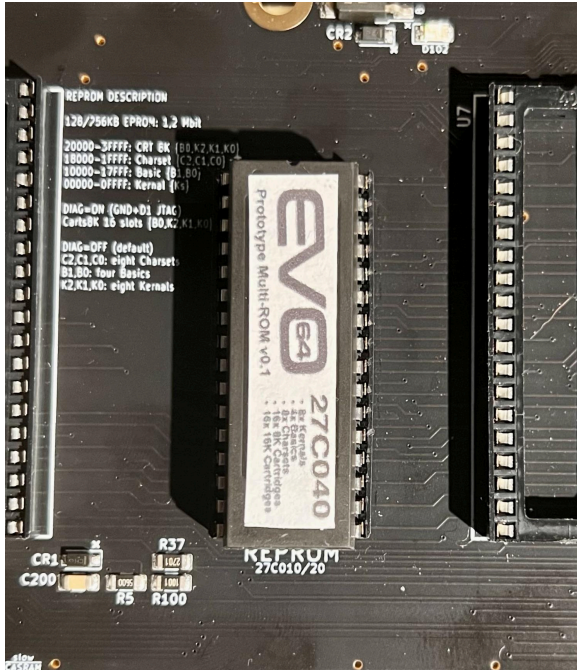
EVO64 EPROM Memory Map

Please note that the colored columns on the left hand side of the page identify the different EPROM types supported by EVO. The length of the column indicates how many ROM / Cart slots can be programmed and supported by each ROM type.

Please also take note of the “**JTAG CRT EN**” column, as this indicates whether EVO's Cartridge Enable jumper needs to be in place for the specific ROM image to be enabled (located on the JTAG pin header bank)

Supported EPROM(S)	ROM Type	ROM Count	ROM Images	ROM Size (KB)	Cumulative Size (KB)	EPROM Mem Addr.	JTAG CRT EN	RePROM Jumpers							
								C2	C1	C0	B1	B0	K2	K1	K0
   	Kernal	1	Standard (901227)	8	8	\$00000 - \$01FFF	00	-	-	-	-	-	0	0	0
		2	Standard (901227)	8	16	\$02000 - \$03FFF	00	-	-	-	-	-	0	0	1
		3	Standard (901227)	8	24	\$04000 - \$05FFF	00	-	-	-	-	-	0	1	0
		4	Standard (901227)	8	32	\$06000 - \$07FFF	00	-	-	-	-	-	0	1	1
		5	Standard (901227)	8	40	\$08000 - \$09FFF	00	-	-	-	-	-	1	0	0
		6	Standard (901227)	8	48	\$0A000 - \$0BFFF	00	-	-	-	-	-	1	0	1
		7	Standard (901227)	8	56	\$0C000 - \$0DFFF	00	-	-	-	-	-	1	1	0
		8	Standard (901227)	8	64	\$0E000 - \$0FFFF	00	-	-	-	-	-	1	1	1
	Basic	1	Standard (901226)	8	72	\$10000 - \$11FFF	00	-	-	-	0	0	-	-	-
		2	Standard with Multiply Fix	8	80	\$12000 - \$13FFF	00	-	-	-	0	1	-	-	-
		3	OpenROM Basic	8	88	\$14000 - \$15FFF	00	-	-	-	1	0	-	-	-
		4	Standard (901226)	8	96	\$16000 - \$17FFF	00	-	-	-	1	1	-	-	-
	Charset	1	Standard (901225)	4	100	\$18000 - \$18FFF	00	0	0	0	-	-	-	-	-
		2	C16/Plus4 Chagen	4	104	\$19000 - \$19FFF	00	0	0	1	-	-	-	-	-
		3	PXL font 88665b	4	108	\$1A000 - \$1AFFF	00	0	1	0	-	-	-	-	-
		4	MSX Chagen	4	112	\$1B000 - \$1BFFF	00	0	1	1	-	-	-	-	-
		5	Atari 800 Chagen	4	116	\$1C000 - \$1CFFF	00	1	0	0	-	-	-	-	-
		6	Amiga 1200	4	120	\$1D000 - \$1DFFF	00	1	0	1	-	-	-	-	-
		7	OpenROM Chagen	4	124	\$1E000 - \$1EFFF	00	1	1	0	-	-	-	-	-
		8	Standard (901225)	4	128	\$1F000 - \$1FFFF	00	1	1	1	-	-	-	-	-
	Cartridge	1	(Empty)	8	136	\$20000 - \$21FFF	01	0	0	0	0	0	0	0	0
		2	Diag Test 586220++	8	144	\$22000 - \$23FFF	01	0	0	0	0	0	0	0	1
		3	64 Dr. Test Keyboard	8	152	\$24000 - \$25FFF	01	0	0	0	0	0	0	1	0
		4	Commodore 64 Diagnostic	8	160	\$26000 - \$27FFF	01	0	0	0	0	0	0	1	1
		5	C-64 Diagnostic Rev 4.1.0	8	168	\$28000 - \$29FFF	01	0	0	0	0	0	1	0	0
		6	SID Test Cartridge (w/Cyberoid II SID)	8	176	\$2A000 - \$2BFFF	01	0	0	0	0	0	1	0	1
		7	The Tool-64 Micro Application	8	184	\$2C000 - \$2DFFF	01	0	0	0	0	0	1	1	0
		8	1541 Diag Test Cart (Jani)	8	192	\$2E000 - \$2FFFF	01	0	0	0	0	0	1	1	1
		9	The Tool-64 Micro Application	8	200	\$30000 - \$31FFF	01	0	0	0	0	1	0	0	0
		10	Frogger	8	208	\$32000 - \$33FFF	01	0	0	0	0	1	0	0	1
		11	Galaxian	8	216	\$34000 - \$35FFF	01	0	0	0	0	1	0	1	0
		12	Mario's Brewery (Donkey Kong Clone)	8	224	\$36000 - \$37FFF	01	0	0	0	0	1	0	1	1
		13	Pacman (83)	8	232	\$38000 - \$39FFF	01	0	0	0	0	1	1	0	0
		14	Pitfall (84)	8	240	\$3A000 - \$3BFFF	01	0	0	0	0	1	1	0	1
		15	Space Action (Defender Clone)	8	248	\$3C000 - \$3DFFF	01	0	0	0	0	1	1	1	0
		16	Tanks	8	256	\$3E000 - \$3FFFF	01	0	0	0	0	1	1	1	1
	Cartridge	17	Bubble Burst	16	272	\$40000 - \$43FFF	01	0	0	0	1	0	0	0	0
		18	Buck Rogers	16	288	\$44000 - \$47FFF	01	0	0	0	1	0	0	0	1
		19	Congo Bongo	16	304	\$48000 - \$4BFFF	01	0	0	0	1	0	0	1	0
		20	Decathlon	16	320	\$4C000 - \$4FFFF	01	0	0	0	1	0	0	1	1
		21	Defender	16	336	\$50000 - \$53FFF	01	0	0	0	1	0	1	0	0
		22	Frogger II	16	352	\$54000 - \$57FFF	01	0	0	0	1	0	1	0	1
		23	Frogger II	16	368	\$58000 - \$5BFFF	01	0	0	0	1	0	1	1	0
		24	Munchman & Galxians	16	384	\$5C000 - \$5FFFF	01	0	0	0	1	0	1	1	1
		25	LaZarian	16	400	\$60000 - \$63FFF	01	0	0	0	1	1	0	0	0
		26	MS. Pacman	16	416	\$64000 - \$67FFF	01	0	0	0	1	1	0	0	1
		27	Adventure 1 - The Mutant Spiders	16	432	\$68000 - \$6BFFF	01	0	0	0	1	1	0	1	0
		28	Popeye	16	448	\$6C000 - \$6FFFF	01	0	0	0	1	1	0	1	1
		29	(Empty)	16	464	\$70000 - \$73FFF	01	0	0	0	1	1	1	0	0
		30	Pitstop	16	480	\$74000 - \$77FFF	01	0	0	0	1	1	1	0	1
		31	Atari Star Wars	16	496	\$78000 - \$7BFFF	01	0	0	0	1	1	1	1	0
		32	Wizard of War	16	512	\$7C000 - \$7FFFF	01	0	0	0	1	1	1	1	1

Changing your EPROM Type



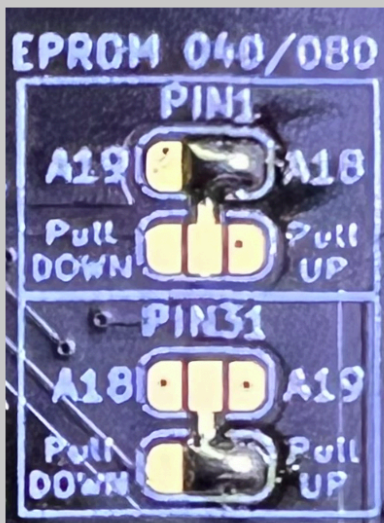
EVO Rev1 boards ship with a **512K AM29F040B** EPROM, which is capable of supporting up to:

- 8x Kernel ROM images @ 8KB each
- 4x Basics @ 8K each
- 8x Character sets @ 4K each
- 16x Cartridge ROM images @ 8K each
- 16x Cartridge ROM images @ 16K each

Other supported EPROM types include:

- 27C010 @ 128K
- 27C020 @ 256K
- 27C040 and 29F040B @ 512K
- 27C080 or 27C801 @ 1MB

The EPROM type selection is made on the underside of the EVO Rel 1 mainboard, by soldering (shorting) a set of jumper-pads that correspond to the EPROM type you intend to use, as outlined in this image:



27C801 PIN1=A19, PIN31=A18
 27C080 PIN1=A19, PIN31=A18
 27C040 PIN1=X, PIN31=A18
 29F040 PIN1=A18, PIN31=PullUP

27C010 = 128 KB
 27C020 = 256 KB
 27C040 or 29F040 = 512 KB
 27C080 or 27C801 = 1 MB